

What is Method?

How method is declared?

Return Type

Access Specifier

Method name and its convention

Parameters

Method Body

Exception thrown (will cover in Excn Handling Topic)

- Types of Method

System defined Method

User defined Method

Overloaded Method

Overridden Method

Static Method

Final Method

Abstract Method

- Variable Arguments (VarArgs) parameter of the Method

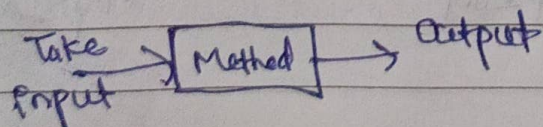
- Memory Allocation of Method (cover in Memory mgmt Topic)

Q. What is Method?

- Method is used to perform certain tasks

- It's a collection of instⁿ that perform some specific task.

- It can be used to bring the code readability and re-usability



// How to call a method.

```

Q. int sumOfDigits(int n) {
    int sum = 0;
    while(n > 0) {
        sum += n % 10;
        n /= 10;
    }
    return sum;
}
  
```

```

obj.sumOfDigits(123);
// 6
  
```

How to Declare method :

⇒ `public returnType methodName (list of para) throws Exception`
`{`
`// method body`
`}`

Access Specifiers :

- `public` : can be access through any class in any package. (collection of logical/similar classes) ^{grouped them together}
- `private` : can be access by methods only in the same class.
- `protected` : can be access by other classes in same package or other sub-classes in diff package. _{child classes}
- `Default` : if we do not mention anything, then Default access specifier is used by Java. It can be only accessed by classes in same package.

Return type

- Method do not return anything use "void"
- use class name or primitive datatype as return type of the method.

Method Name

- It should be verb (some kind of action)
- Should start with small letter and follow camel case in case of multiple words

Method Body

- Method body get finished when you call "return" in ^{mid}
- Not finished when reached to the end
- You can also stop method by "return" even for void return type.

Types of Methods

- i) **System defined Methods** : Methods which are already defined and ready to use in Java like `Math.sqrt()`, `Math.log10(N)`;
 - JRE contains library classes where all SDM is defined.
- ii) **User Defined Method** :
 - Methods which programmer creates based upon the program necessity.

Ex: `int doSum(int a, int b) {`
 `return a+b;`

① }
 }
 }

iii) Overloaded Method: (Static binding)

- More than one method but different parameters and having the same name is created in same class.

`int doSum(int[] arr) {`

~~return~~ `int sum = 0;`

`for (int x : arr) {`

`sum += x;`

`}`

`return sum;`
`}`

both ① & ②
are overloaded
method imagin
both present in same
class

iv) Overridden Method: (Dynamic binding)

- Subclass has the same method as the parent class.

① `void getInvoice() {`

`Sout("Invoice of a Person");`

`}`
`}`

return type, argument, name are same only
method definition is changed a/c to
business requirement.

```
class Employee extends Person {
    @Override
    void getInvoice() {
        sout("get Invoice of Employee");
    }
}
```

both ① & ② are overridden method because the definition of getInvoice is updated in child class

* Static Method: ★★

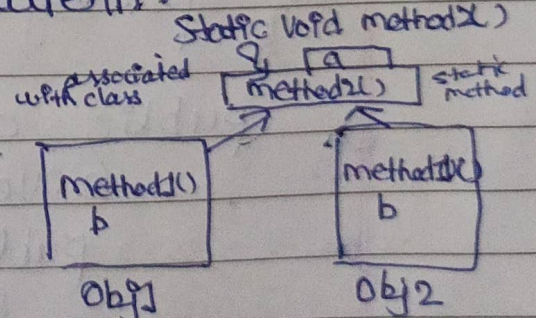
- These are associated with the class
- can be called just with class name
- Static methods can not access Non Static Instance Variable and methods.
- Static method can not be overridden, it just hide it

When to declare method static:

- Methods which do not modify the state of the object can be declared static.
- Utility Method which do not use any instance variable and compute only on arguments.

Example: Factory design pattern.

```
class AS {
    static A;
    void method1();
    static void method2();
}
```



* Final Method

- Public class Calculation {

static int compPrice = 40;

public static int Sum(int a, int b) {

return a+b; }

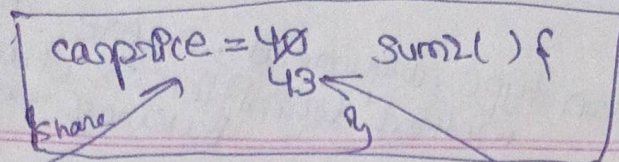
public static int sum2(int a, int b) {

int total = a+b;

compPrice = total;

return compPrice;

Obj1.method1(); // call
A.method2(); // call static method.



thread1
Sum2(2,1) // 43

thread2
Sum2(5,2) // 7+43 = 50

we expect 47 but get 50 due to presence of instance variable in static method

Final Method :

- Final Method cannot be overridden in Java
- Why? final method means its implementation can not be changed. If child class cannot change its implementation then no use of overridden

Abstract Method :

- It is defined only in Abstract classes.
- Only method declaration is done
- Its implementation is done in child classes

```
public abstract class Person {
    public final void profession() {
        sout("hi my child class can't override me")
    }
}
```

```
public abstract int print(); // method declaration only. defn provide by child class who extends me
```

```
public class Employee extends Person {
    @Override
```

```
    public void profession() {
        sout("No child class can override final Method");
    }
```

error X
Not possible

```
    public int print() {
        // do something
        return 0;
    }
```

Variable Arguments (VARARGS): ★★

- Variable number of inputs in the parameter.
- Only one variable argument can be present in the method.
- It should be the last argument in the list.

Ex: class A {

```
    public int sum( int a, int ... variables) {
```

```
        int output += a;
```

```
        for( int var : variables) {
```

```
            output += var;
```

```
        }
```

```
    } return output;
```

```
}
```

```
class Main {
```

```
    psum() {
```

```
        A obj = new A();
```

```
        A.sum(1); // 1
```

```
        A.sum(1, 2); // 3
```

```
        A.sum(1, 2, 5, 9, 10, 11, 12); // 50
```

```
    }
```

```
}
```